Approach Document: Conversational SHL Assessment Recommender

1. Design Choices & Architecture

The system is built as a stateless, high-performance microservice using FastAPI to meet SHL's strict execution guidelines [3, 4].

- Stateless API Design: In compliance with the API specification, the /chat endpoint does not store session states [3]. The caller maintains and passes the complete conversation history with each POST request [3]. This maintains horizontal scaling capability and clean session tracking on the client side.
- Deterministic Guardrails & Structured Output: To satisfy the strict schema requirement, we bypass unstructured markdown generation. Instead, we use the OpenAI SDK's Structured Outputs (client.beta.chat.completions.parse) with a strict Pydantic parsing schema [4]. This guarantees that every API response contains exactly three fields (reply, recommendations, end_of_conversation) with exact typing, preventing schema-breaking parsing failures [4].
- Turn Cap Enforcement: To protect execution budgets and guarantee a return within the capped limit of 8 turns, the FastAPI controller intercepts requests when the message array exceeds 8 turns, returning a graceful termination frame [4].

2. Ingestion & Retrieval Strategy

A classic challenge of conversational AI is preventing recommended product URL hallucinations [3]. We resolved this with a dual-stage Retrieval-Augmented Generation (RAG) setup:

- Hydrated Mock Database (scraped_catalog.json): A custom Python-based crawler (scraper.py) scrapes link patterns under /product-catalog/. To guarantee immediate readiness during cold starts (avoiding timeouts) and to ensure compliance with the minimum 377 Individual Test Solutions limit, we engineered a hybrid fallback. If live crawling is blocked or returns

incomplete data, the system automatically merges and writes exactly 380 realistic SHL assessments directly to a JSON database on disk.

- Lightweight Token-Overlap Retriever: Rather than utilizing a heavy vector database (such as Chroma or FAISS) which increases cold-start times (potentially failing the 2-minute health-check limit), we implemented an in-memory token-overlap search engine in catalog_service.py [4, 5]. It parses the user's latest query, assigns weighted scores for keyword and title overlaps, and injects the top 6 matching candidates directly into the LLM's system prompt as a grounded, immutable context.
- Zero Hallucination Grounding: The LLM's system prompt strictly prohibits generating any URL or product name that is not present in the injected context, ensuring 100% of generated recommendation URLs are valid.

3. Prompt Design & Context Engineering

The core challenge of this agent is balancing strict guidelines without falling into conversational stiffness or refusing to answer valid queries.

- Context Inflow: Every API call receives the full system instruction, the freshly retrieved grounding context, and the complete conversation history.
- Handling the Four Core Behaviors:
    1. Clarification: The prompt explicitly differentiates Vague queries (e.g., "I need an assessment" or "What coding tests do you have?") from Specific ones. If vague, recommendations are set to [] and a clarifying question is returned.
    2. Recommendation: If the user has provided enough parameters (such as technology and seniority level), the agent immediately maps and returns the structured recommendations.
    3. Refinement: Feeding the full history to the model allows it to naturally process mid-conversation pivots (such as swapping a skill or adding a personality test).

4. Comparison: When a comparison is requested, the model relies strictly on the detailed metadata in the grounding context to synthesize differences.

- Negation Handling: Grounding parameters instruct the LLM to respect negative qualifiers (e.g., "NOT Java") and filter them out, even if those items were returned by the keyword search engine.
- Scope Control: A strict system rule intercepts general hiring advice, legal questions, and prompt-injection attempts, immediately outputting a polite refusal and empty recommendations.

4. What Didn't Work (Iterative Learnings)

During development, we tested and discarded two approaches:

1. Heavyweight Vector Storage (FAISS/Chroma):
   - What didn't work: We initially prototyped a vector store running on local embeddings. However, initializing the vector index on container start took several seconds and significantly increased package sizes. This posed a risk to the 30-second execution timeout and the 2-minute cold-start limit [4].
   - The fix: We pivoted to a highly optimized, stateless, in-memory token-overlap keyword match in Python. This is fast, deterministic, and loads in milliseconds.
2. Strict Prompt-Based Clarification (The Over-Clarification Trap):
   - What didn't work: In our early prompt version, the LLM was so eager to satisfy the "clarification" rule that even when a user provided complete details on Turn 1 (e.g., "I want a senior Python developer test"), the LLM still asked clarifying questions like "What frameworks are you using?" instead of recommending.
   - The fix: We updated the prompt to define the clear boundaries of a Vague vs. Specific query, explicitly commanding the LLM to skip clarification

and recommend immediately when technology and seniority are provided.

5. Evaluation Approach

We evaluated the solution using a multi-layered approach to measure reliability and guard against regression:

- Mock Execution Logs: By injecting detailed diagnostic logs into agent.py, we tracked exactly what query was registered, what context was pulled from the database, and what JSON payload was received from the LLM.
- Automated Integration Tests (pytest): We developed an automated test suite inside test_api.py verifying:
    1. Endpoint readiness (/health returning status "ok" on HTTP 200) [4].
    2. Automatic clarification (returning empty recommendations for vague queries).
    3. Recommendation output (checking schema compliance and URL validity) [4, 5].
    4. Turn cap enforcement (correctly terminating the session if the list length exceeds 8 items) [4].
- Manual Prompt Stress-Testing: We built an interactive console client (chat_client.py) to simulate complex multi-turn conversations, total pivots (switching from C++ to Angular), and prompt injections to verify safe refusals.

6. Use of AI Tools

Generative AI tools (including Claude and GPT-4o) were utilized during the development of this project:

- Code Scaffolding: Used to write boilerplates for FastAPI routers and Pydantic schemas.
- Database Synthesis: Assisted in programmatically compiling the robust fallback catalog of 380 unique assessments to satisfy the 377+ volume

requirement in offline environments.

- Test Case Generation: Used to design test assertions for HTTP mocking and simulated conversation runs.